

Threads Übungen

ThreadUtil

Schreibe eine Methode **sleep** für eine Hilfsklasse **ThreadUtil**, die das Schlafen eines Threads für eine bestimmte Dauer ermöglicht, ohne dass der aufrufende Code die InterruptedException behandeln muss.

Verwende die Klasse `java.time.Duration` als **Parameter**, um die Zeitdauer anzugeben.

Die Methode `sleep` soll intern `Thread.sleep(...)` verwenden, dabei aber die Checked Exception (InterruptedException) in eine Unchecked Exception umwandeln.

Beispielverwendung:

```
public static void main(String[] args) {  
    ThreadUtil.sleep(Duration.ofSeconds(1)); // no try-catch required.  
}
```

Dog Eat Cat

Schreibe eine App, die folgende drei Threads startet:

1. Gibt für immer "Dog " jede Sekunde auf der Konsole aus.
2. Gibt für immer "Eat " jede Sekunde auf der Konsole aus.
3. Gibt für immer "Cat " jede Sekunde auf der Konsole aus.

StringAdder

Schreibe zwei Threads **A** und **B**.

A und B schreiben in **dieselbe** Liste.

A fügt **x-Mal (limit) "a"** zur Liste hinzu. **Nach jedem Hinzufügen soll 10 ms gewartet werden.**

B fügt **x-Mal (limit) "b"** zur Liste hinzu. **Nach jedem Hinzufügen soll 10 ms gewartet werden.**

Starte A und B als Threads in folgender Methode:

`Thread.sleep(10);`

`static List<String> compute(int limit) throws InterruptedException` *// returns the result of the shared list*

Schreibe einen **Unit Test**, der das Ergebnis von **compute(10)** testet.

```
class ListThreads {  
  
    static List<String> compute(int limit) throws InterruptedException {  
        List<String> sharedList;  
        // TODO: erzeuge Threads für A und B und starte Threads, die beide die sharedList befüllen.  
        return sharedList;  
    }  
}
```

ThreadSafeList

Gegeben ist folgender Code.

Ändere nur die Klasse ThreadSafeList, sodass der Unit Test "grün" wird.

Du muss **synchronized** verwenden!

```
class ThreadSafeList<E> extends AbstractList<E> {

    private final List<E> elements = new LinkedList<>();

    @Override
    public boolean add(E element) {
        elements.addLast(element);
        return true;
    }

    @Override
    public int size() {
        return elements.size();
    }

    @Override
    public E get(int index) {
        return elements.get(index);
    }

    @Override
    public String toString() {
        return "ThreadSafeList[" + elements + "]";
    }
}
```

Read-only

```
class FixThreadSafeListTest {

    @RepeatedTest(5)
    void readWriteListShouldNotThrowAnyException() throws InterruptedException {
        boolean exceptionWasThrown = FixThreadSafeList.readWriteList();
        assertFalse(exceptionWasThrown);
    }
}
```

Read-only

```
class FixThreadSafeList {
    private static final int TEN_TIMES = 10;

    static boolean readWriteList() throws InterruptedException {
        List<Integer> threadSafeList = new ThreadSafeList<>();
        threadSafeList.add(0);

        boolean[] exceptionWasThrown = {false};

        var aThread = new Thread(() -> {
            for (int i = 1; i <= TEN_TIMES; i++) {
                try {
                    threadSafeList.add(i);
                } catch (Exception e) {
                    e.printStackTrace();
                    exceptionWasThrown[0] = true;
                }
                try { Thread.sleep(10); } catch (InterruptedException e) { throw new RuntimeException(e); }
            }
        });

        var bThread = new Thread(() -> {
            for (int i = 1; i <= TEN_TIMES; i++) {
                try {
                    System.out.println(threadSafeList);
                } catch (Exception e) {
                    e.printStackTrace();
                    exceptionWasThrown[0] = true;
                }
                try { Thread.sleep(10); } catch (InterruptedException e) { throw new RuntimeException(e); }
            }
        });

        aThread.start();
        bThread.start();

        aThread.join();
        bThread.join();

        return exceptionWasThrown[0];
    }

    public static void main(String[] args) throws InterruptedException {
        readWriteList();
    }
}
```

SynchronizedArrayList

Ändere die folgende Klasse so, dass sie thread-safe ist.

```
/**
 * Change this class so that it is thread-safe.
 */
class SynchronizedArrayList<T> {

    private final ArrayList<T> elements = new ArrayList<>();

    public boolean add(T element) {
        return elements.add(element);
    }

    public int size() {
        return elements.size();
    }
}
```

Drone

Gegeben ist folgender Code.

Ändere ihn, sodass er thread-safe ist.

```
final class App {  
  
    private static final int NUMBER_OF_DRONES = 100;  
  
    public static void main(String[] args) throws InterruptedException {  
        try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {  
            ChargingStation centralOne = new ChargingStation();  
            for (int id = 1; id <= NUMBER_OF_DRONES; id++) {  
                executor.submit(new Drone(id, centralOne));  
            }  
            executor.awaitTermination(Long.MAX_VALUE, TimeUnit.SECONDS);  
        }  
    }  
}
```

```
class ChargingStation {  
  
    private int numberOfLandedDrones = 0;  
  
    boolean isOccupied() {  
        return numberOfLandedDrones != 0;  
    }  
  
    void land() {  
        numberOfLandedDrones++;  
        checkNumberOfLandedDrones();  
    }  
  
    void takeOff() {  
        numberOfLandedDrones--;  
        checkNumberOfLandedDrones();  
    }  
  
    private void checkNumberOfLandedDrones() {  
        if (numberOfLandedDrones < 0 || numberOfLandedDrones > 1) {  
            System.err.printf("numberOfLandedDrones must be between 0 and 1 but was %d%n", numberOfLandedDrones);  
            System.exit(404);  
        }  
    }  
}
```

```
class Drone implements Runnable {  
  
    private final int id;  
    private final ChargingStation chargingStation;  
  
    private int battery = 100;  
  
    Drone(int id, ChargingStation chargingStation) {  
        this.id = id;  
        this.chargingStation = chargingStation;  
    }  
  
    void fly() {  
        battery -= 10;  
        if (battery <= 10) {  
            emergencyLand();  
        } else if (battery <= 30) {  
            tryLand();  
        }  
    }  
  
    void tryLand() {  
        if (!chargingStation.isOccupied()) {  
            chargingStation.land();  
            recharge();  
            chargingStation.takeOff();  
        }  
    }  
  
    private void emergencyLand() {  
        battery = 0;  
    }  
  
    void recharge() {  
        battery = 100;  
    }  
  
    @Override  
    public void run() {  
        while (battery > 0) {  
            fly();  
        }  
    }  
  
    @Override  
    public String toString() {  
        return "%s (%d)".formatted(id, battery);  
    }  
}
```